



# Week 7 Workshop Evaluation

**IFN647**

Tutor

School of Computer Science  
Queensland University of Technology

# Workshop Objectives

- Develop a clear understanding of the fundamental evaluation metrics in Information Retrieval (IR), including **Precision**, **Recall**, and the **F1-measure**, as well as rank-based metrics such as **Precision@n** and **Recall@n** for evaluating ranked retrieval results.
- Gain the ability to systematically use **relevance judgements** (ground-truth labels indicating whether documents are relevant to a query) to assess the effectiveness of IR models.
- Build practical skills by implementing Python programs that:
  - Compute **Precision**, **Recall**, and **F1-measure** based on model outputs and relevance judgements.
  - Evaluate ranked retrieval results by calculating **Precision@k** and **Recall@k** at various cutoff positions.
  - Identify rank positions where relevant documents are retrieved, compute Precision at those positions, and use these values to calculate **Average Precision (AP)** for a query.

# Workshop Tasks

- Task 1:**

Understand and explain **Precision**, **Recall**, and **F1-Measure** with their formulas and relevance to IR evaluation.

- Task 2:**

Write a Python program to **read relevance data**, compute **Precision**, **Recall**, and **F1-Measure** based on retrieved vs relevant documents.

- Task 3:**

Extend the program to handle **ranked retrieval output** and calculate **Average Precision** based on relevant documents' positions in the ranked list.

# Workshop Task 1.1

**Task 1.** Understand the definitions of Precision, recall and F measure and discuss how to use the following definitions. You may discuss these measures with your classmate or your tutor if you have any doubts.

(1) Assume  $A$  is a set of relevant documents (e.g., a benchmark, or relevance judgments),  $B$  is a set of retrieved documents (e.g., the output of an IR model, a binary output)

- **binary\_output.txt**

```
R105 2493 1
R105 2494 0
R105 3008 1
R105 5004 0
R105 5223 0
R105 5225 1
R105 5226 1
R105 6635 0
R105 7937 0
R105 15744 1
R105 17650 0
R105 22961 0
R105 25263 0
R105 26711 0
R105 27966 0
R105 28198 0
```

- **relevance\_judgment.txt**

```
R105 2493 1
R105 2494 1
R105 3008 1
R105 5004 1
R105 5223 0
R105 5225 1
R105 5226 1
R105 6635 0
R105 7937 0
R105 15744 1
R105 17650 0
R105 22961 0
R105 25263 0
R105 26711 0
R105 27966 0
R105 28198 0
```

Check rel\_data Folder for data files

Format:

***Topic* DocumentID Relevance**

# Workshop Task 1.1

**Task 1.** Understand the definitions of Precision, recall and F measure and discuss how to use the following definitions. You may discuss these measures with your classmate or your tutor if you have any doubts.

- (1) Assume  $A$  is a set of relevant documents (e.g., a benchmark, or relevance judgments),  $B$  is a set of retrieved documents (e.g., the output of an IR model, a binary output)

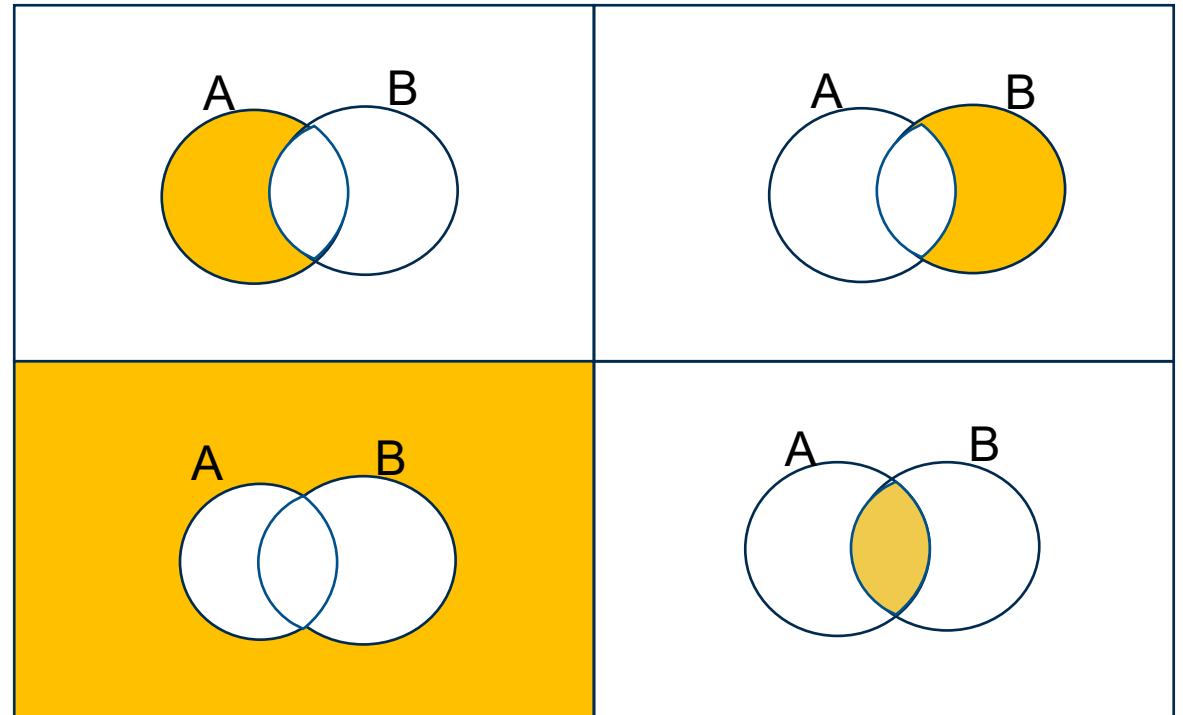
|                      | $A$ (Relevant)              | $\bar{A}$ (Non-Relevant)          |
|----------------------|-----------------------------|-----------------------------------|
| $B$ (Retrieved)      | (1) $A \cap B$ (✓ TP)       | (3) $\bar{A} \cap B$ (✗ FP)       |
| $\bar{B}$ (Not Ret.) | (2) $A \cap \bar{B}$ (✗ FN) | (4) $\bar{A} \cap \bar{B}$ (✓ TN) |

$A$  is a set of relevant documents

$\bar{A}$  is a set of non-relevant documents

$B$  is a set of retrieved documents

$\bar{B}$  is a set of non-retrieved documents



# Workshop Task 1.1

**Task 1.** Understand the definitions of Precision, recall and F measure and discuss how to use the following definitions. You may discuss these measures with your classmate or your tutor if you have any doubts.

- (1) Assume  $A$  is a set of relevant documents (e.g., a benchmark, or relevance judgments),  $B$  is a set of retrieved documents (e.g., the output of an IR model, a binary output)

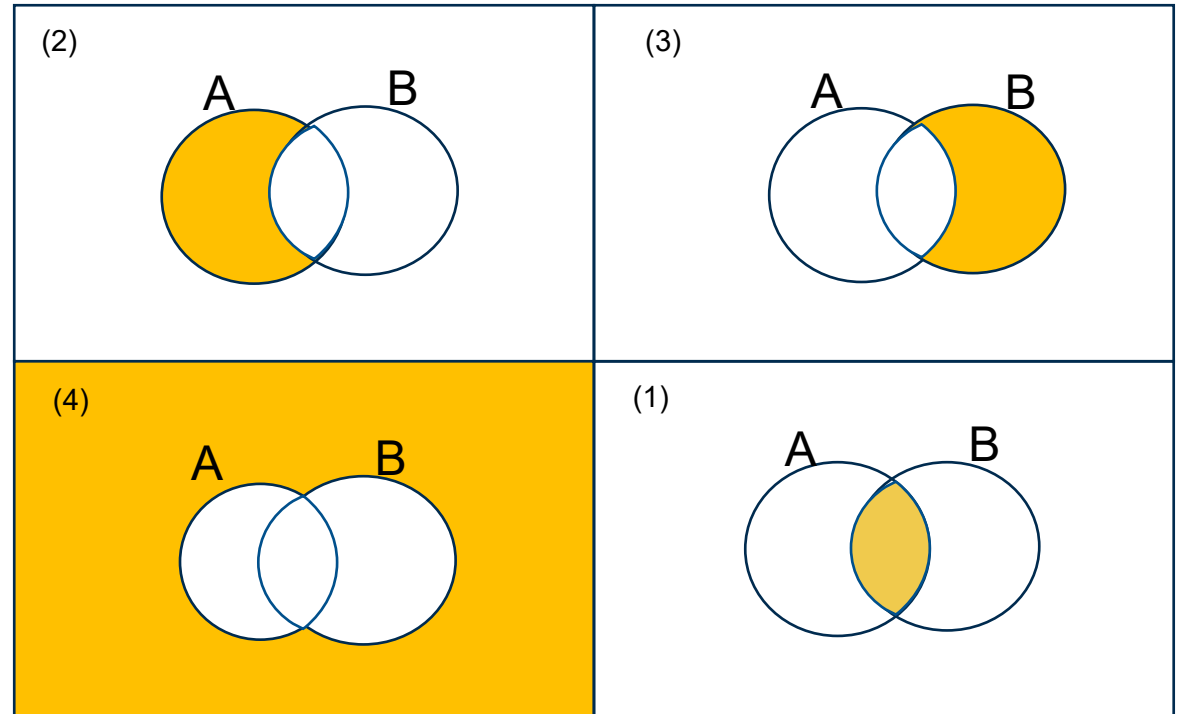
|                      | $A$ (Relevant)              | $\bar{A}$ (Non-Relevant)          |
|----------------------|-----------------------------|-----------------------------------|
| $B$ (Retrieved)      | (1) $A \cap B$ (✓ TP)       | (3) $\bar{A} \cap B$ (✗ FP)       |
| $\bar{B}$ (Not Ret.) | (2) $A \cap \bar{B}$ (✗ FN) | (4) $\bar{A} \cap \bar{B}$ (✓ TN) |

$A$  is a set of relevant documents

$\bar{A}$  is a set of non-relevant documents

$B$  is a set of retrieved documents

$\bar{B}$  is a set of non-retrieved documents



# Workshop Task 1.1

**Task 1.** Understand the definitions of Precision, recall and F measure and discuss how to use the following definitions. You may discuss these measures with your classmate or your tutor if you have any doubts.

- (1) Assume  $A$  is a set of relevant documents (e.g., a benchmark, or relevance judgments),  $B$  is a set of retrieved documents (e.g., the output of an IR model, a binary output)

|                      | A (Relevant)                | $\bar{A}$ (Non-Relevant)          |
|----------------------|-----------------------------|-----------------------------------|
| B (Retrieved)        | (1) $A \cap B$ (✓ TP)       | (3) $\bar{A} \cap B$ (✗ FP)       |
| $\bar{B}$ (Not Ret.) | (2) $A \cap \bar{B}$ (✗ FN) | (4) $\bar{A} \cap \bar{B}$ (✓ TN) |

$$Recall = \frac{|A \cap B|}{|A|}$$

$$Precision = \frac{|A \cap B|}{|B|}$$

F-Measure (F1),

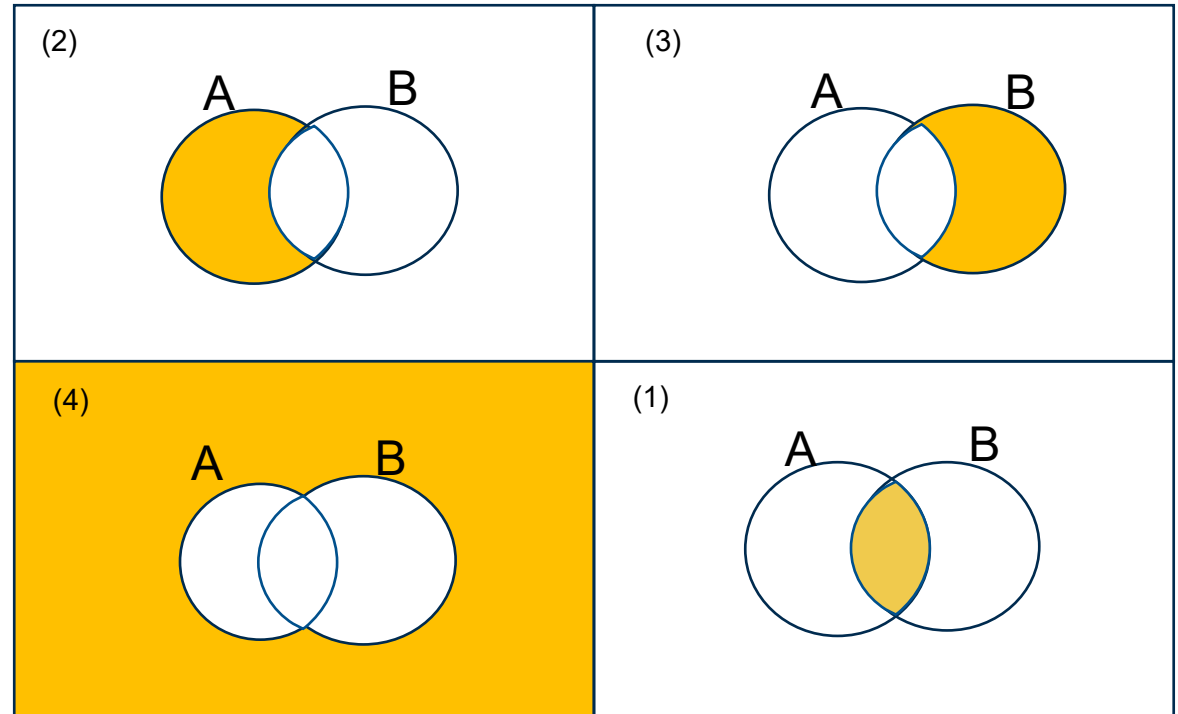
$$F = \frac{1}{\frac{1}{2}(\frac{1}{R} + \frac{1}{P})} = \frac{2RP}{(R+P)}$$

$A$  is a set of relevant documents

$\bar{A}$  is a set of non-relevant documents

$B$  is a set of retrieved documents

$\bar{B}$  is a set of non-retrieved documents



# Workshop Task 1.1

**Task 1.** Understand the definitions of Precision, recall and F measure and discuss how to use the following definitions. You may discuss these measures with your classmate or your tutor if you have any doubts.

- (1) Assume  $A$  is a set of relevant documents (e.g., a benchmark, or relevance judgments),  $B$  is a set of retrieved documents (e.g., the output of an IR model, a binary output)

|                      | $A$ (Relevant)              | $\bar{A}$ (Non-Relevant)          |
|----------------------|-----------------------------|-----------------------------------|
| $B$ (Retrieved)      | (1) $A \cap B$ (✓ TP)       | (3) $\bar{A} \cap B$ (✗ FP)       |
| $\bar{B}$ (Not Ret.) | (2) $A \cap \bar{B}$ (✗ FN) | (4) $\bar{A} \cap \bar{B}$ (✓ TN) |

$$\text{Recall} = \frac{|A \cap B|}{|A|}$$

Out of all actual relevant, how many were correctly predicted as relevant

$$\text{Precision} = \frac{|A \cap B|}{|B|}$$

Out of all predicted as relevant, how many were actually relevant?

F-Measure (F1),

$$F = \frac{1}{\frac{1}{2}(\frac{1}{R} + \frac{1}{P})} = \frac{2RP}{(R+P)}$$

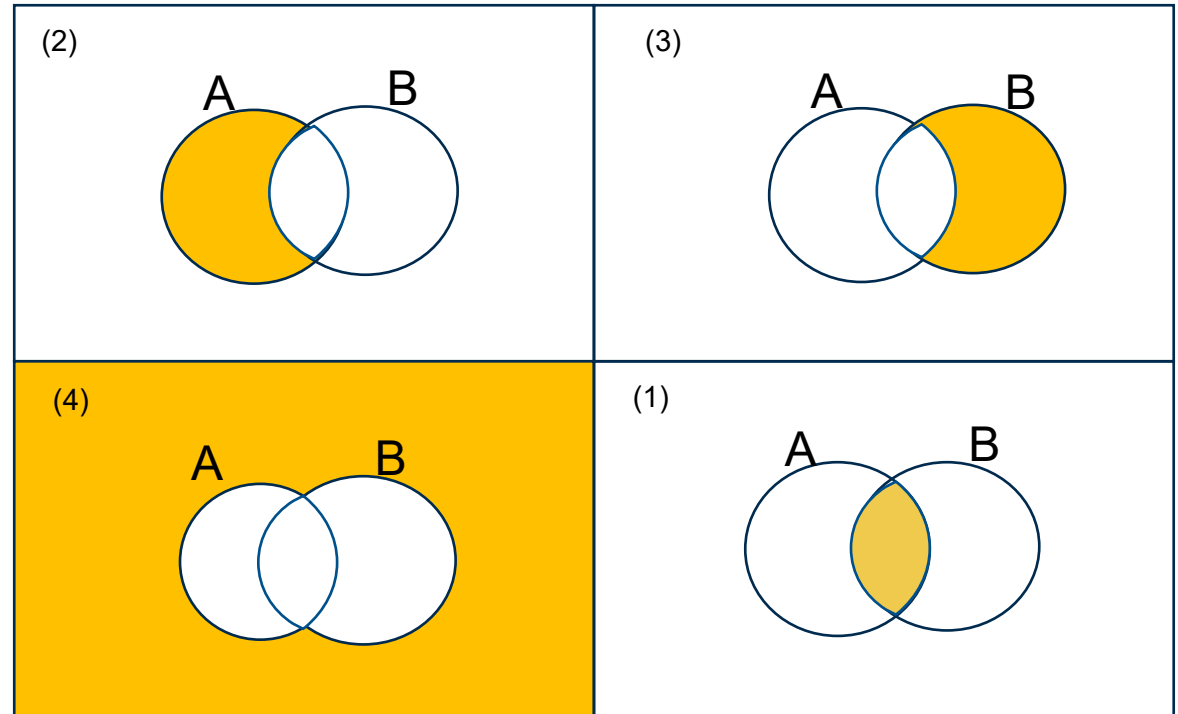
The F-measure gives equal weight to precision and recall, and it is useful when both measures are important.

$A$  is a set of relevant documents

$\bar{A}$  is a set of non-relevant documents

$B$  is a set of retrieved documents

$\bar{B}$  is a set of non-retrieved documents





# Workshop Task 1.2

- ranked\_output.txt

(2) Assume an IR model returns a list of ranked documents (in descending order by the document weight or score). At each position  $n$ , we have the following definition of precision (recall) at position  $n$ :

$$Recall@n = \frac{|A \cap B_n|}{|A|}$$

Of all the relevant documents, how many did we retrieve in the top  $n$ ?

$$Precision@n = \frac{|A \cap B_n|}{n}$$

Of the top  $n$  documents the model returned, how many were actually relevant?

where  $B_n$  is the set of **top- $n$**  documents in the output of the IR model.

```
R105 2493 1.0
R105 3008 0.9
R105 5225 0.8
R105 5226 0.7
R105 15744 0.6
R105 31141 0.6
R105 40239 0.5
R105 48148 0.5
R105 49633 0.4
R105 51493 0.4
R105 59813 0.3
R105 65210 0.3
R105 80484 0.3
R105 86961 0.3
R105 2494 0.2
R105 5004 0.19
R105 5223 0.18
R105 6635 0.17
R105 7937 0.16
R105 17650 0.15
R105 22961 0.14
R105 25263 0.13
```

These definitions are useful (e.g., for calculating average precisions) for IR models that rank documents

# Precision@k and Recall@k

$$Recall@n = \frac{|A \cap B_n|}{|A|}$$

Of all the relevant documents, how many did we retrieve in the top n?

$$Precision@n = \frac{|A \cap B_n|}{n}$$

Of the top n documents the model returned, how many were actually relevant?

Imagine your IR model returns this list of documents ranked by relevance score:

|      |       |     |              |
|------|-------|-----|--------------|
| R105 | 2493  | 1.0 | ✗            |
| R105 | 3008  | 0.9 | ✗            |
| R105 | 5225  | 0.8 | ✓ (relevant) |
| R105 | 5226  | 0.7 | ✗            |
| R105 | 15744 | 0.6 | ✗            |

 Task: Calculate Precision@5 and Recall@5

$Recall@5 = 1/4 = 0.25$  (In the top 5 documents, we retrieved 25% of all relevant documents)

$Precision@5 = 1/5 = 0.2$  (Only 20% of the top-5 results were relevant.)

“Even though the model gave us 5 results, most weren’t helpful early on.”

Relevant documents (A):

R105 5225, R105 31141, R105 40239, R105 51493

$|A| = 4$  relevant documents.

# Why precision and recall alone are not enough

$$\text{Recall}@n = \frac{|A \cap B_n|}{|A|}$$

Of all the relevant documents, how many did we retrieve in the top n?

$$\text{Precision}@n = \frac{|A \cap B_n|}{n}$$

Of the top n documents the model returned, how many were actually relevant?

Imagine your IR model returns this list of documents ranked by relevance score:

|      |       |     |              |
|------|-------|-----|--------------|
| R105 | 2493  | 1.0 | ✗            |
| R105 | 3008  | 0.9 | ✗            |
| R105 | 5225  | 0.8 | ✓ (relevant) |
| R105 | 5226  | 0.7 | ✗            |
| R105 | 15744 | 0.6 | ✗            |
| R105 | 31141 | 0.6 | ✓ (relevant) |
| R105 | 40239 | 0.5 | ✓ (relevant) |
| R105 | 48148 | 0.5 | ✗            |
| R105 | 49633 | 0.4 | ✗            |
| R105 | 51493 | 0.4 | ✓ (relevant) |

Relevant documents (A):

R105 5225, R105 31141, R105 40239, R105 51493

$|A|$  = 4 relevant documents.

 Task: Calculate Precision@5 and Recall@5

$$\text{Recall}@5 = 1/4 = 0.25$$

(We retrieved 25% of all relevant documents in the top-5.)

$$\text{Precision}@5 = 1/5 = 0.2$$

(Only 20% of the top-5 results were relevant.)

“Even though the model gave us 5 results, most weren’t helpful early on.”

 Task: Calculate Precision@10 and Recall@10

$$\text{Recall}@10 = 4/4 = 1.0$$

“We found all the relevant docs — but spread across the top 10.”

$$\text{Precision}@10 = 4/10 = 0.4$$

“Only 40% of the top-10 documents are relevant.”

“This shows how recall can be perfect — but if the relevant docs are buried deep in the list, precision drops”. - So recall looks perfect, but the system is not actually good

# Why need to calculate average precisions

“A good IR model should not just find relevant documents — it should find them *early* in the ranking. That’s why we need both precision and recall.”

“AP rewards systems that retrieve relevant documents higher in the ranking”

why **Average Precision (AP)** is a better metric when we care about both:

- **early relevance** (relevant docs high in the list)
- **completeness** (all relevant docs found eventually)

|      |       |     |   |                             |
|------|-------|-----|---|-----------------------------|
| R105 | 2493  | 1.0 | ✗ |                             |
| R105 | 3008  | 0.9 | ✗ |                             |
| R105 | 5225  | 0.8 | ✓ | (relevant) Precision = 1/3  |
| R105 | 5226  | 0.7 | ✗ |                             |
| R105 | 15744 | 0.6 | ✗ |                             |
| R105 | 31141 | 0.6 | ✓ | (relevant) Precision = 2/6  |
| R105 | 40239 | 0.5 | ✓ | (relevant) Precision = 3/7  |
| R105 | 48148 | 0.5 | ✗ |                             |
| R105 | 49633 | 0.4 | ✗ |                             |
| R105 | 51493 | 0.4 | ✓ | (relevant) Precision = 4/10 |

$$AP = \frac{1/3 + 2/6 + 3/7 + 4/10}{4} = \text{around } 0.373$$

|      |       |     |   |            |
|------|-------|-----|---|------------|
| R105 | 5225  | 1.0 | ✓ | (relevant) |
| R105 | 31141 | 0.9 | ✓ | (relevant) |
| R105 | 40239 | 0.8 | ✓ | (relevant) |
| R105 | 51493 | 0.7 | ✓ | (relevant) |
| R105 | ----- | 0.6 |   |            |
| R105 | ----- | 0.6 |   |            |
| R105 | ----- | 0.5 |   |            |
| R105 | ----- | 0.5 |   |            |
| R105 | ----- | 0.4 |   |            |
| R105 | -     | 0.4 |   |            |

Compare this to another model where relevant docs are at ranks 1, 2, 3, 4:

$$AP = \frac{1/1 + 2/2 + 3/3 + 4/4}{4} = 1.0$$

Even though both models found all relevant documents (Recall@10 = 1.0), **AP tells us which one did it earlier and better.**

# Workshop Task 2

## INPUT

### • relevance\_judgment.txt

```
R105 2493 1
R105 2494 1
R105 3008 1
R105 5004 1
R105 5223 0
R105 5225 1
R105 5226 1
R105 6635 0
R105 7937 0
R105 15744 1
R105 17650 0
R105 22961 0
R105 25263 0
R105 26711 0
R105 27966 0
R105 28198 0
R105 31141 0
R105 33961 0
R105 38229 0
R105 40239 1
R105 40259 1
...
```

### • binary\_output.txt

```
R105 2493 1
R105 2494 0
R105 3008 1
R105 5004 0
R105 5223 0
R105 5225 1
R105 5226 1
R105 6635 0
R105 7937 0
R105 15744 1
R105 17650 0
R105 22961 0
R105 25263 0
R105 26711 0
R105 27966 0
R105 28198 0
```

**Task 2:** read a topic-doc-assignment file (e.g., *relevance\_judgments.txt*, the benchmark) and a retrieved topic-doc-assignment file (e.g., *binary\_output.txt*, the output of an IR model for query *R105*); and calculate the IR model's Recall, Precision, and F-Measure (F1).

- Please download two topic-doc-assignment files (*rel\_data.zip*) and save them in a folder (e.g., “rel\_data”). Both files have the format of

***Topic DocumentID Relevance***

For the file “relevance\_judgments.txt”, *Relevance* (Relevance Judgment) = “1” indicates relevant and “0” means non-relevant. For the file “binary\_output.txt”, the *relevance* values are generated by the IR model. We can obtain a set of retrieved documents by selecting the rows with *Relevance* (Relevance Values) = “1”.

- Define a function *rel\_setting(inputpath)*, which reads the two topic-doc-assignment files in the folder *inputpath*, and returns a pair of dictionaries {*documentID: Relevance\_judgment, ...*} and {*documentID: Relevance\_value, ...*} for all documents in “relevance\_judgments.txt”, and “binary\_output.txt”, respectively.
- Define a main function to call function *rel\_setting()*, calculate Recall, Precision, and F-measure and display the result.



# Workshop Task 2

## INPUT

|                                 |                            |
|---------------------------------|----------------------------|
| • <b>relevance_judgment.txt</b> | • <b>binary_output.txt</b> |
| R105 2493 1                     | R105 2493 1                |
| R105 2494 1                     | R105 2494 0                |
| R105 3008 1                     | R105 3008 1                |
| R105 5004 1                     | R105 5004 0                |
| R105 5223 0                     | R105 5223 0                |
| R105 5225 1                     | R105 5225 1                |
| R105 5226 1                     | R105 5226 1                |
| R105 6635 0                     | R105 6635 0                |
| R105 7937 0                     | R105 7937 0                |
| R105 15744 1                    | R105 15744 1               |
| R105 17650 0                    | R105 17650 0               |
| R105 22961 0                    | R105 22961 0               |
| R105 25263 0                    | R105 25263 0               |
| R105 26711 0                    | R105 26711 0               |
| R105 27966 0                    | R105 27966 0               |
| R105 28198 0                    | R105 28198 0               |
| R105 31141 0                    |                            |
| R105 33961 0                    |                            |
| R105 38229 0                    |                            |
| R105 40239 1                    |                            |
| R105 40259 1                    |                            |
| ...                             |                            |

## OUTPUT

```
Rel_doc = {'2493': 1, '2494': 1, '3008': 1, '5004': 1,
'5223': 0, '5225': 1, '5226': 1, '6635': 0, '7937': 0,
'15744': 1, '17650': 0, '22961': 0, '25263': 0,
'26711': 0, '27966': 0, '28198': 0, '31141': 0,
'33961': 0, ...}
```

```
Ret_doc = {'2493': 1, '2494': 0, '3008': 1, '5004': 0,
'5223': 0, '5225': 1, '5226': 1, '6635': 0, '7937': 0,
'15744': 1, '17650': 0, '22961': 0, '25263': 0, ...}
```

The number of relevant documents: 16

The number of retrieved documents: 14

The number of retrieved documents that are relevant: 11

recall = 0.6875

precision = 0.7857142857142857

F-Measure = 0.7333333333333334

**Task 2:** read a topic-doc-assignment file (e.g., *relevance\_judgments.txt*, the benchmark) and a retrieved topic-doc-assignment file (e.g., *binary\_output.txt*, the output of an IR model for query *R105*); and calculate the IR model's Recall, Precision, and F-Measure (F1).

- Please download two topic-doc-assignment files (*rel\_data.zip*) and save them in a folder (e.g., “rel\_data”). Both files have the format of

**Topic DocumentID Relevance**

For the file “relevance\_judgments.txt”, *Relevance* (Relevance Judgment) = “1” indicates relevant and “0” means non-relevant. For the file “binary\_output.txt”, the *relevance* values are generated by the IR model. We can obtain a set of retrieved documents by selecting the rows with *Relevance* (Relevance Values) = “1”.

- Define a function *rel\_setting(inputpath)*, which reads the two topic-doc-assignment files in the folder *inputpath*, and returns a pair of dictionaries {*documentID: Relevance\_judgment, ...*} and {*documentID: Relevance\_value, ...*} for all documents in “relevance\_judgments.txt”, and “binary\_output.txt”, respectively.
- Define a main function to call function *rel\_setting()*, calculate Recall, Precision, and F-measure and display the result.

- relevance\_judgment.txt

- binary\_output.txt



```
Rel_doc = {'2493': 1, '2494': 1, '3008': 1, '5004': 1,  
'5223': 0, '5225': 1, '5226': 1, '6635': 0, '7937': 0,  
'15744': 1, '17650': 0, '22961': 0, '25263': 0,  
'26711': 0, '27966': 0, '28198': 0, '31141': 0,  
'33961': 0, ...
```

```
Ret_doc = {'2493': 1, '2494': 0, '3008': 1, '5004': 0,  
'5223': 0, '5225': 1, '5226': 1, '6635': 0, '7937': 0,  
'15744': 1, '17650': 0, '22961': 0, '25263': 0, ...
```



The number of relevant documents: 16

The number of retrieved documents: 14

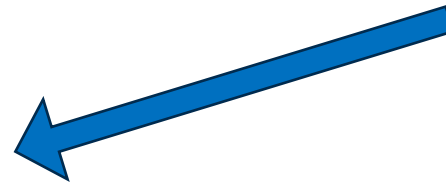
The number of retrieved documents that are relevant: 11



$$Recall = \frac{|A \cap B|}{|A|}$$

$$Precision = \frac{|A \cap B|}{|B|}$$

$$F = \frac{1}{\frac{1}{2}(\frac{1}{R} + \frac{1}{P})} = \frac{2RP}{(R+P)}$$



recall = 0.6875

precision = 0.7857142857142857

F-Measure = 0.7333333333333334

# Sample Task2 Solutions

```
A = Rel_doc = {'2493': 1, '2494': 1, '3008': 1,
'5004': 1, '5223': 0, '5225': 1, '5226': 1, '6635': 0,
'7937': 0, '15744': 1, '17650': 0, '22961': 0,
'25263': 0, '26711': 0, '27966': 0, '28198': 0,
'31141': 0, '33961': 0, ...}
```

```
B = Ret_doc = {'2493': 1, '2494': 0, '3008': 1, '5004':
0, '5223': 0, '5225': 1, '5226': 1, '6635': 0, '7937':
0, '15744': 1, '17650': 0, '22961': 0, '25263': 0, ...}
```

```
def wk7_input(inputpath):
    coll = {}
    os.chdir(inputpath)
    # for Task 2
    A = {}
    B = {}
```

Save relevance judgments and relevance values(retrieved or not)

```
# for Task 2
for line in open("relevance_judgments.txt"):
    line = line.strip()
    line1 = line.split()
    A[line1[1]] = int(float(line1[2]))
for line in open("binary_output.txt"):
    line = line.strip()
    line1 = line.split()
    B[line1[1]] = int(float(line1[2]))
# for Task 3
```

Read each file in the rel\_data.zip and save data as dictionaries

```
return (A,B,R)
```

Return three dictionaries, A, B (R for Task 3, we'll talk later).

• relevance\_judgment.txt

```
R105 2493 1
R105 2494 1
R105 3008 1
R105 5004 1
R105 5223 0
R105 5225 1
R105 5226 1
R105 6635 0
R105 7937 0
R105 15744 1
R105 17650 0
R105 22961 0
R105 25263 0
R105 26711 0
R105 27966 0
R105 28198 0
R105 31141 0
R105 33961 0
R105 38229 0
R105 40239 1
R105 40259 1
...
```

• binary\_output.txt

```
R105 2493 1
R105 2494 0
R105 3008 1
R105 5004 0
R105 5223 0
R105 5225 1
R105 5226 1
R105 6635 0
R105 7937 0
R105 15744 1
R105 17650 0
R105 22961 0
R105 25263 0
R105 26711 0
R105 27966 0
R105 28198 0
```



# Sample Task2 Solutions

```
if __name__ == '__main__':
    import sys
    if len(sys.argv) != 2:
        sys.stderr.write("USAGE: %s <coll-file>\n" % sys.argv[0])
        sys.exit()
    (rel_doc, retrived_doc, ranked_doc) = wk7_input(sys.argv[1])

    # for task 2
    R = 0
    for (x,y) in rel_doc.items():
        if (y==1):
            R= R+1
    print("The number of relevant documents: " + str(R))

    R1 = 0
    for (x,y) in retrived_doc.items():
        if (y==1):
            R1= R1+1
    print("The number of retrieved documents: "+ str(R1))

    RR1 = 0
    for (x,y) in retrived_doc.items():
        if (y==1) & (rel_doc[x]==1):
            RR1= RR1+1

    print("The number of retrieved documents that are relevant: " + str(RR1))
    r = float(RR1)/float(R)
    p = float(RR1)/float(R1)
    F1 = 2*p*r/(p+r)
    print("recall = " + str(r))
    print("precision = " + str(p))
    print("F-Measure = " + str(F1))
```

Call the function with three input files and get returned dictionaries

Count the number of relevant documents based on the returned relevant judgement dictionary rel\_doc.

compute the number of retrieved documents according to the returned dictionary retireved\_doc.

Count the number of retrieved relevant documents

Calculate the recall, precision and F1 based on their formulations

```
Rel_doc = {'2493': 1, '2494': 0, '3008': 1, '5004': 1,
'5223': 0, '5225': 1, '5226': 1, '6635': 0, '7937': 0,
'15744': 1, '17650': 0, '22961': 0, '25263': 0,
'26711': 0, '27966': 0, '28198': 0, '31141': 0,
'33961': 0, ...}
```

```
Ret_doc = {'2493': 1, '2494': 1, '3008': 1, '5004': 0,
'5223': 0, '5225': 1, '5226': 1, '6635': 0, '7937': 0,
'15744': 1, '17650': 0, '22961': 0, '25263': 0, ...}
```

|               | Relevant                                  | Non-Relevant                                          |
|---------------|-------------------------------------------|-------------------------------------------------------|
| Retrieved     | $\frac{A \cap B}{ A \cap B }$             | $\frac{A \cap \bar{B}}{ A \cap \bar{B} }$             |
| Not Retrieved | $\frac{\bar{A} \cap B}{ \bar{A} \cap B }$ | $\frac{\bar{A} \cap \bar{B}}{ \bar{A} \cap \bar{B} }$ |

$$Recall = \frac{|A \cap B|}{|A|}$$

$$Precision = \frac{|A \cap B|}{|B|}$$

$$F = \frac{1}{\frac{1}{2}(\frac{1}{R} + \frac{1}{P})} = \frac{2RP}{(R+P)}$$

# Sample Task2 Results

The number of relevant documents: 16

The number of retrieved documents: 14

The number of retrieved documents that are relevant: 11

recall = 0.6875

precision = 0.7857142857142857

F-Measure = 0.7333333333333334

|               | Relevant         | Non-Relevant           |
|---------------|------------------|------------------------|
| Retrieved     | $A \cap B$       | $\bar{A} \cap B$       |
| Not Retrieved | $A \cap \bar{B}$ | $\bar{A} \cap \bar{B}$ |

$$Recall = \frac{|A \cap B|}{|A|}$$

$$Precision = \frac{|A \cap B|}{|B|}$$

$$F = \frac{1}{\frac{1}{2}(\frac{1}{R} + \frac{1}{P})} = \frac{2RP}{(R+P)}$$

# Workshop Task 3

**Task 3:** Read the ranked output file (“ranked\_output.txt”, the ranked output of another IR model for query *R105*), and calculate its average precision.

- Please download the ranked output file and save it in the same folder you created for Task 2.
- Extend function `rel_setting(inputpath)` to return 3 dictionaries (two for task 2 and one for task 3). The dictionary for task 3 should have the format of `{rankingNo: documentID, ...}`, and it only includes top-10 documents (ranked in descending order), where  $rankingNo = 1, 2, \dots, 10$ .
- Extend the main function to calculate Recall and Precision at the rank positions where a relevant document was retrieved, and then calculate the average precision, and print out the result.

- ranked\_output.txt

```
R105 2493 1.0
R105 3008 0.9
R105 5225 0.8
R105 5226 0.7
R105 15744 0.6
R105 31141 0.6
R105 40239 0.5
R105 48148 0.5
R105 49633 0.4
R105 51493 0.4
R105 59813 0.3
R105 65210 0.3
R105 80484 0.3
R105 86961 0.3
R105 2494 0.2
R105 5004 0.19
R105 5223 0.18
R105 6635 0.17
R105 7937 0.16
R105 17650 0.15
R105 22961 0.14
R105 25263 0.13
```

# Workshop Task 3

```
C = {'2493': 1.0, '3008': 0.9, '5225': 0.8, '5226': 0.7, '15744': 0.6, '31141': 0.6, '40239': 0.5, '48148': 0.5, '49633': 0.4, '51493': 0.4, '59813': 0.3, '65210': 0.3, ...}
```

## Example of output

```
R = {1: '2493', 2: '3008', 3: '5225', 4: '5226', 5: '15744', 6: '31141', 7: '40239', 8: '48148', 9: '49633', 10: '51493'}
```

```
At position 1docID: 2493, precision= 1.0
At position 2docID: 3008, precision= 1.0
At position 3docID: 5225, precision= 1.0
At position 4docID: 5226, precision= 1.0
At position 5docID: 15744, precision= 1.0
At position 7docID: 40239, precision= 0.8571428571428571
At position 8docID: 48148, precision= 0.875
At position 9docID: 49633, precision= 0.8888888888888888
At position 10docID: 51493, precision= 0.9
The average precision = 0.9467813051146384
```

- ranked\_output.txt

```
R105 2493 1.0
R105 3008 0.9
R105 5225 0.8
R105 5226 0.7
R105 15744 0.6
R105 31141 0.6
R105 40239 0.5
R105 48148 0.5
R105 49633 0.4
R105 51493 0.4
R105 59813 0.3
R105 65210 0.3
R105 80484 0.3
R105 86961 0.3
R105 2494 0.2
R105 5004 0.19
R105 5223 0.18
R105 6635 0.17
R105 7937 0.16
R105 17650 0.15
R105 22961 0.14
R105 25263 0.13
```

$$Recall@n = \frac{|A \cap B_n|}{|A|}$$

$$Precision@n = \frac{|A \cap B_n|}{n}$$

# Sample Task3 Solutions

```
def wk7_input(inputpath):
    coll = {}
    os.chdir(inputpath)
    # for Task 2
    A = {}
    B = {}
    # for Task 3
    C = {}
    R = {}

    # for Task 2
    for line in open("relevance_judgments.txt"):
        line = line.strip()
        line1 = line.split()
        A[line1[1]] = int(float(line1[2]))
    for line in open("binary_output.txt"):
        line = line.strip()
        line1 = line.split()
        B[line1[1]] = int(float(line1[2]))
    # for Task 3
    for line in open("ranked_output.txt"):
        line = line.strip()
        line1 = line.split()
        C[line1[1]] = float(line1[2])

    # get the top-10 document in terms of {rankingNO: documentID, ...}
    i=1
    for (k,v) in sorted(C.items(), key=lambda x:x[1], reverse=True):
        R[i] = k
        i = i+1
        if i>10:
            break
    #print(sorted(C.items(), key=lambda x: float(x[1])))
    print(R)
    return (A,B,R)
```

Save relevance judgments and relevance values(retrieved or not)

Save the ranked output and the top-10 ranked documents

Read each file in the rel\_data.zip and save data as dictionaries • ranked\_output.txt

Sort the ranking value in descending order and get the top-10 document in terms of {rankingNo: documentID, ...}

Return three dictionaries, A, B and R.

```
C = {'2493': 1.0, '3008': 0.9, '5225': 0.8, '5226': 0.7, '15744': 0.6, '31141': 0.6,
'40239': 0.5, '48148': 0.5, '49633': 0.4, '51493': 0.4, '59813': 0.3, '65210': 0.3,...}

R = {1: '2493', 2: '3008', 3: '5225', 4: '5226', 5: '15744', 6: '31141', 7:
'40239', 8: '48148', 9: '49633', 10: '51493'}
```

```
R105 2493 1.0
R105 3008 0.9
R105 5225 0.8
R105 5226 0.7
R105 15744 0.6
R105 31141 0.6
R105 40239 0.5
R105 48148 0.5
R105 49633 0.4
R105 51493 0.4
R105 59813 0.3
R105 65210 0.3
R105 80484 0.3
R105 86961 0.3
R105 2494 0.2
R105 5004 0.19
R105 5223 0.18
R105 6635 0.17
R105 7937 0.16
R105 17650 0.15
R105 22961 0.14
R105 25263 0.13
```

# Sample Task3 Solutions

```
ranked_doc = R = {1: '2493', 2: '3008', 3: '5225', 4: '5226', 5: '15744', 6: '31141', 7: '40239', 8: '48148', 9: '49633', 10: '51493'}
```

```
Rel_doc = {'2493': 1, '2494': 0, '3008': 1, '5004': 1, '5223': 0, '5225': 1, '5226': 1, '6635': 0, '7937': 0, '15744': 1, '17650': 0, '22961': 0, '25263': 0, '26711': 0, '27966': 0, '28198': 0, '31141': 0, '33961': 0, ...}
```

```
(rel_doc, retrived_doc, ranked_doc) = wk7_input(sys.argv[1])
```

```
# code for task 3
```

```
print("For task 3:")
```

```
ri = 0 Count the number of retrieved relevant documents
```

```
ap1 = 0.0 To get the sum of all the precision values calculated at position n.
```

```
for (n,id) in sorted(ranked_doc.items(), key=lambda x: int(x[0])):
```

```
    if (rel_doc[id]==1):
```

```
        ri = ri+1
```

```
        pi = float(ri)/float(int(n))
```

```
        ap1 = ap1 + pi
```

```
        print("At position " + str(int(n)) + "docID: " + id + ", precision= " + str(pi))
```

```
ap1 = ap1/float(ri)
```

```
print("The average precision = " + str(ap1))
```

$$Recall@n = \frac{|A \cap B_n|}{|A|}$$

$$Precision@n = \frac{|A \cap B_n|}{n}$$

where  $B_n$  is the set of **top- $n$**  documents in the output of the IR model.

Contain top-10 documents in term of  
{rankingNo: documentID, ...}

Count the number of retrieved relevant  
documents

Calculate the precision value at position  $n$ .

Count the total precision values

Only calculate the average  
precision for those retrieved  
relevant documents.

# Sample Task3 Results

For task 3:

```
At position 1docID: 2493, precision= 1.0
At position 2docID: 3008, precision= 1.0
At position 3docID: 5225, precision= 1.0
At position 4docID: 5226, precision= 1.0
At position 5docID: 15744, precision= 1.0
At position 7docID: 40239, precision= 0.8571428571428571
At position 8docID: 48148, precision= 0.875
At position 9docID: 49633, precision= 0.8888888888888888
At position 10docID: 51493, precision= 0.9
The average precision = 0.9467813051146384
```

Of the top 10 documents the model returned, on average 95% are actually relevant.